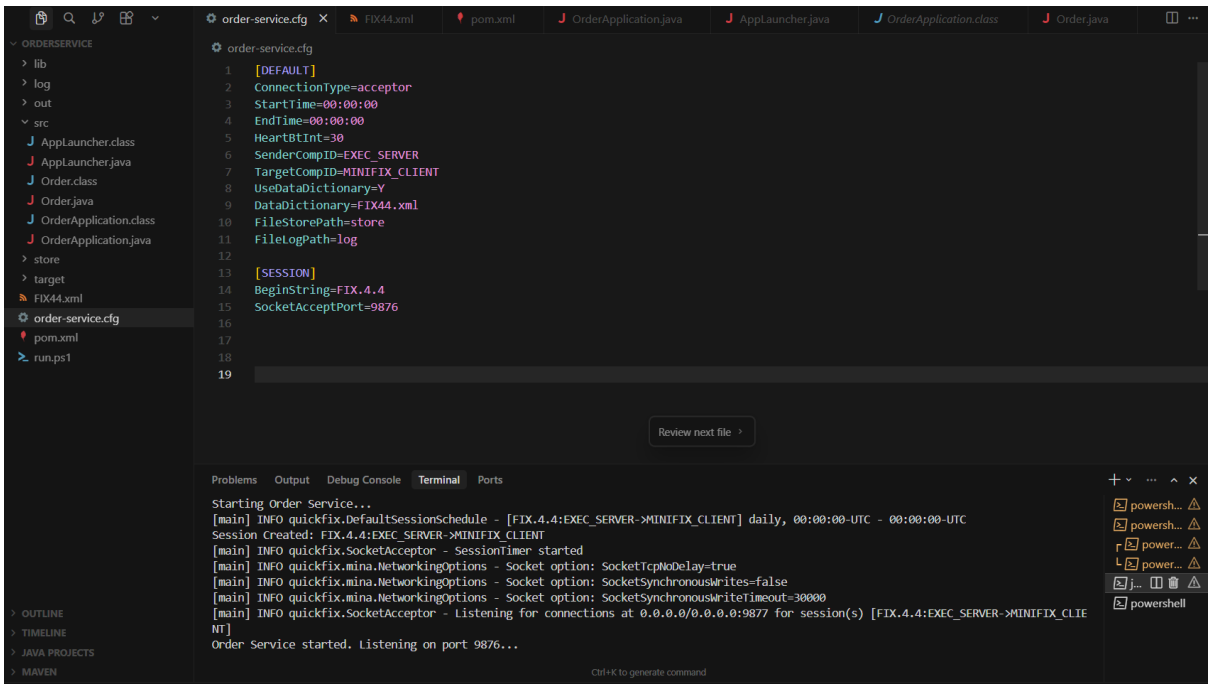
Ayush Das CML Week 2 Submission

~230958124

~Roll No. 56

Q.1 order-service.cfg file



Q2. Part-1 Logon - MINIFIX_CLIENT

```
>> javac -cp $cp -d out src\*.java
PS C:\Users\ayush\Documents\OrderService> java -cp "out;$cp" AppLauncher
[main] INFO quickfix.DefaultSessionSchedule - [FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT] daily, 00:00:00-UTC - 00:00:00-UTC
<20260221-16:47:17, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, event> (Session FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT schedule is daily, 00:0
0:00-UTC - 00:00:00-UTC)
<20260221-16:47:17, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, event> (Created session: FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT)
Session Created: FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT
[main] INFO quickfix.SocketAcceptor - SessionTimer started
[main] INFO quickfix.mina.NetworkingOptions - Socket option: SocketTcpNoDelay=true
[main] INFO quickfix.mina.NetworkingOptions - Socket option: SocketSynchronousWrites=false
[main] INFO quickfix.mina.NetworkingOptions - Socket option: SocketSynchronousWriteTimeout=30000
[main] INFO quickfix.SocketAcceptor - Listening for connections at 0.0.0.0/0.0.0.0:9876 for session(s) [FIX.4.4:EXEC_SERVER->MINIFIX
_CLIENT]
Order Service started. Listening on port 9876...
[QFJ Message Processor] INFO quickfix.SocketAcceptor - Started QFJ Message Processor
[NioProcessor-2] INFO quickfix.mina.acceptor.AcceptorIoHandler - MINA session created: local=/127.0.0.1:9876, class org.apache.mina.
transport.socket.nio.NioSocketSession, remote=/127.0.0.1:52010
<20260221-16:47:27, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, incoming> (8=FIX.4.49=7635=A49=MINIFIX_CLIENT56=EXEC_SERVER34=152=20260221-
16:47:2798=0108=3010=204)
<20260221-16:47:27, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, event> (Accepting session FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT from /127.0.0
.1:52010)
<20260221-16:47:27, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, event> (Acceptor heartbeat set to 30 seconds)
<20260221-16:47:27, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, event> (Received logon)
<20260221-16:47:27, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, event> (Responding to Logon request)
<20260221-16:47:27, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, outgoing> (8=FIX.4.49=8035=A34=149=EXEC_SERVER52=20260221-16:47:27.13956=MI
NIFIX_CLIENT98=0108=3010=146)
LOGON Success: FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT
<20260221-16:47:41, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, incoming> (8=FIX.4.49=6435=049=MINIFIX_CLIENT56=EXEC_SERVER34=252=20260221-
16:47:4110=156)
<20260221-16:47:57, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, outgoing> (8=FIX.4.49=6835=034=249=EXEC_SERVER52=20260221-16:47:57.73556=MI
NIFIX_CLIENT10=116)
<20260221-16:48:11, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, incoming> (8=FIX.4.49=6435=049=MINIFIX_CLIENT56=EXEC_SERVER34=352=20260221-
16:48:1110=155)
<20260221-16:48:27, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, outgoing> (8=FIX.4.49=6835=034=349=EXEC_SERVER52=20260221-16:48:27.73456=MI
NIFIX_CLIENT10=114)
```

Q2. Part-2 subsequent Heartbeat messages

```
<20260221-16:47:41, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, incoming> (8=FIX.4.49=6435=049=MINIFIX_CLIENT56=EXEC_SERVER34=252=20260221-
16:47:4110=156)
<20260221-16:47:57, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, outgoing> (8=FIX.4.49=6835=034=249=EXEC_SERVER52=20260221-16:47:57.73556=MI
NIFIX_CLIENT10=116)
<20260221-16:48:11, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, incoming> (8=FIX.4.49=6435=049=MINIFIX_CLIENT56=EXEC_SERVER34=352=20260221-
16:48:1110=155)
<20260221-16:48:27, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, outgoing> (8=FIX.4.49=6835=034=349=EXEC_SERVER52=20260221-16:48:27.73456=MI
NIFIX_CLIENT10=114)
<20260221-16:48:41, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, incoming> (8=FIX.4.49=6435=049=MINIFIX_CLIENT56=EXEC_SERVER34=452=20260221-
16:48:4110=159)
<20260221-16:48:57, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, outgoing> (8=FIX.4.49=6835=034=449=EXEC_SERVER52=20260221-16:48:57.73656=MI
NIFIX_CLIENT10=120)
<20260221-16:49:11, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, incoming> (8=FIX.4.49=6435=049=MINIFIX_CLIENT56=EXEC_SERVER34=552=20260221-
16:49:1110=158)
<20260221-16:49:27, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, outgoing> (8=FIX.4.49=6835=034=549=EXEC_SERVER52=20260221-16:49:27.73556=MI
NIFIX_CLIENT10=118)
<20260221-16:49:41, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, incoming> (8=FIX.4.49=6435=049=MINIFIX_CLIENT56=EXEC_SERVER34=652=20260221-
16:49:4110=162)
<20260221-16:49:57, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, outgoing> (8=FIX.4.49=6835=034=649=EXEC_SERVER52=20260221-16:49:57.73456=MI
NIFIX_CLIENT10=121)
<20260221-16:50:11, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, incoming> (8=FIX.4.49=6435=049=MINIFIX_CLIENT56=EXEC_SERVER34=752=20260221-
16:50:1110=152)
<20260221-16:50:27, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, outgoing> (8=FIX.4.49=6835=034=749=EXEC_SERVER52=20260221-16:50:27.73556=MI
NIFIX_CLIENT10=112)
<20260221-16:50:41, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, incoming> (8=FIX.4.49=6435=049=MINIFIX_CLIENT56=EXEC_SERVER34=852=20260221-
16:50:4110=156)
<20260221-16:50:57, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, outgoing> (8=FIX.4.49=6835=034=849=EXEC_SERVER52=20260221-16:50:57.73556=MI
NIFIX_CLIENT10=116)
<20260221-16:51:11, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, incoming> (8=FIX.4.49=6435=049=MINIFIX_CLIENT56=EXEC_SERVER34=952=20260221-
16:51:1110=155)
<20260221-16:51:27, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, outgoing> (8=FIX.4.49=6835=034=949=EXEC_SERVER52=20260221-16:51:27.73456=MI
NIFIX_CLIENT10=114)
<20260221-16:51:41, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, incoming> (8=FIX.4.49=6535=049=MINIFIX_CLIENT56=EXEC_SERVER34=1052=20260221
-16:51:4110=199)
<20260221-16:51:57, FIX.4.4:EXEC_SERVER->MINIFIX_CLIENT, outgoing> (8=FIX.4.49=6935=034=1049=EXEC_SERVER52=20260221-16:51:57.73456=M
INIFIX_CLIENT10=158)
```

Q.3. Briefly explain what would happen if you changed the SenderCompID in the simulator but not in your Java code.

If the **SenderCompID in the simulator is changed but not updated in the Java Order Service configuration**, the FIX session will fail to establish successfully.

In FIX protocol, **SenderCompID (Tag 49)** and **TargetCompID (Tag 56)** uniquely identify each counterparty and form the session identity. During the Logon (MsgType = A), the server validates these IDs against its configuration.

If the IDs do not match:

1. The Order Service will detect a session identity mismatch.
2. The Logon request from the simulator may be rejected or immediately followed by a Logout (MsgType = 5).
3. The FIX session will not transition to an active state.
4. Heartbeat messages will not be exchanged, and no trading messages (e.g., NewOrderSingle) can be processed.

As a result, the simulator will display connection errors or repeated logon failures, and the Order Service console will not show **LOGON Success**.

This occurs because FIX relies on strict counterparty identification to ensure message integrity, prevent spoofing, and correctly route orders between trading participants.

Ayush Das CML Week 3 Submission

~230958124

~Roll No. 56

Q.1. processNewOrder method logic code snippet

Code:

```
private void processNewOrder(Message message, SessionID sessionId) {
        try {
            String clOrdId = message.getString(ClOrdID.FIELD);
            String symbol = message.getString(Symbol.FIELD);
            char side = message.getChar(Side.FIELD);
            double qty = message.getDouble(OrderQty.FIELD);
            double price = message.getDouble(Price.FIELD);
            System.out.printf(
                "ORDER RECEIVED: ID=%s Side=%s Sym=%s Px=%.2f Qty=%.0f%n",
                clOrdId, (side == '1' ? "BUY" : "SELL"), symbol, price, qty
            );
            if (qty <= 0 || price <= 0) {
                sendReject(message, sessionId, "Invalid Price or Qty");
            } else {
                acceptOrder(message, sessionId);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
```

Snippet:

```
    private void processNewOrder(Message message, SessionID sessionId) {
        try {
            String clOrdId = message.getString(ClOrdID.FIELD);
            String symbol = message.getString(Symbol.FIELD);
            char side = message.getChar(Side.FIELD);
            double qty = message.getDouble(OrderQty.FIELD);
            double price = message.getDouble(Price.FIELD);

            System.out.printf(
                format: "ORDER RECEIVED: ID=%s Side=%s Sym=%s Px=%.2f Qty=%.0f%n",
                clOrdId, (side == '1' ? "BUY" : "SELL"), symbol, price, qty
            );

            if (qty <= 0 || price <= 0) {
                sendReject(message, sessionId, reason: "Invalid Price or Qty");
            } else {
                acceptOrder(message, sessionId);
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
```

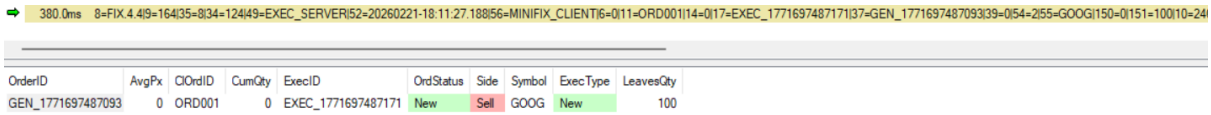
Logic: The `processNewOrder()` method is responsible for handling incoming **NewOrderSingle (MsgType = D)** messages received from the FIX engine.

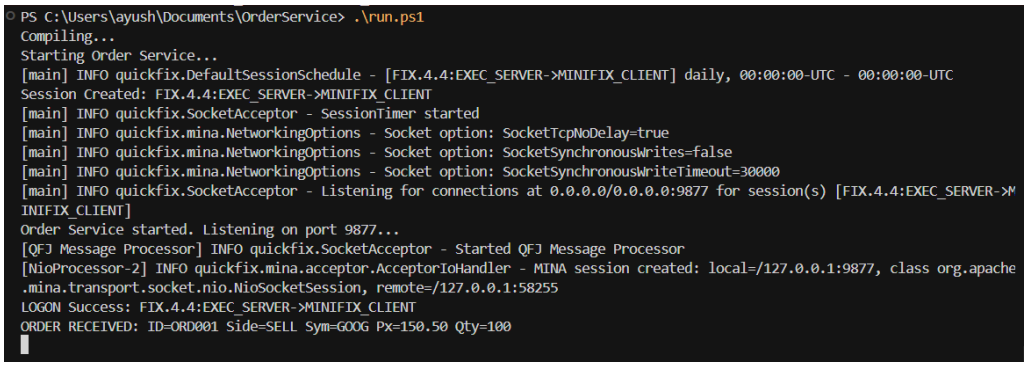
First, the method extracts key order details such as **ClOrdID, Symbol, Side, OrderQty, and Price** from the FIX message using QuickFIX field accessors. These values are then mapped into an internal representation and printed to the console for verification.

Next, the method performs basic validation by checking whether the order quantity and price are positive. If the order fails validation, a reject response is generated. Otherwise, the order is accepted and an **ExecutionReport acknowledgment** is sent back to the client to confirm receipt of the order.

This ensures that every valid order is parsed, validated, and acknowledged, preventing duplicate submissions and maintaining proper order lifecycle tracking.

Q.2. The MiniFix "Trade/Order Blotter" showing orders with status "New" (Confirmed) rather than "Pending".





Q.3. Why do we need to echo back the ClOrdID (Tag 11) in the response? What happens if the Client sends two orders and we swap the IDs in the response?

Why do we need to echo back the ClOrdID (Tag 11) in the response?

The ClOrdID is the unique identifier assigned by the client to each order. Echoing the same ClOrdID in the ExecutionReport allows the client to correctly match the server's response with the original order request. This ensures accurate order tracking, state reconciliation, and prevents duplicate or lost orders in high-frequency trading environments.

What happens if the Client sends two orders and we swap the IDs in the response?

If the ExecutionReports contain incorrect ClOrdIDs, the client will associate acknowledgments with the wrong orders. This can lead to:

- Incorrect order status tracking
- Duplicate order resubmissions (client assumes original order was lost)
- Reconciliation failures between client and server
- Potential trading errors and financial risk

Therefore, preserving and echoing ClOrdID is critical for maintaining message integrity and order lifecycle consistency.

Ayush Das CML Week 5 Submission

~230958124

~Roll No. 56

Q.1 The OrderPersister run loop

```java
public class OrderPersister implements Runnable {

    private final BlockingQueue<Order> queue;
    private volatile boolean running = true;

    public OrderPersister(BlockingQueue<Order> queue) {
        this.queue = queue;
    }

    // ── Runnable ──────────────────────────────────────────────────────────

    @Override
    public void run() {
        System.out.println(x: "[OrderPersister] Persistence worker started.");

        while (running) {
            try {
                // Blocks until an Order is available - zero busy-wait overhead.
                Order order = queue.take();

                boolean saved = DatabaseManager.insertOrder(order);

                if (saved) {
                    System.out.println("[OrderPersister] Persisted order: " + order.getClOrdID()
                                       + " | server_id=" + order.getOrderId());
                }

            } catch (InterruptedException e) {
                // Restore the interrupted flag and let the loop exit cleanly.
                Thread.currentThread().interrupt();
                System.out.println(x: "[OrderPersister] Worker interrupted - shutting down.");
                break;
            }
        }

        System.out.println(x: "[OrderPersister] Worker stopped.");
    }
```

Loop

Q.2 SQL Query with populated data

```
23        quantity   DECIMAL(15,0),
24        status     VARCHAR(20),
25        timestamp  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
26    );
27 •  SELECT * FROM orders;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

order_id	cl_ord_id	symbol	side	price	quantity	status	timestamp
0987ac82-d737-49c5-9b66-876c658af604	ORD-0020	AAPL	1	150.20	100	NEW	2026-03-31 14:09:53
0bdf3249-0b9f-4117-ad65-198fa6229380	ORD-0063	AAPL	1	150.63	100	NEW	2026-03-31 14:09:53
0d6c1545-4f91-43fc-b590-3d33443fdac1	ORD-0059	AAPL	1	150.59	100	NEW	2026-03-31 14:09:53
1013fc1f-8a5f-4e72-96b8-953ceb846346	ORD-0078	AAPL	1	150.78	100	NEW	2026-03-31 14:09:54
105ecb79-9c75-4884-a7cd-0b7f49b8931c	ORD-0068	AAPL	1	150.68	100	NEW	2026-03-31 14:09:54
12be6ffe-cc1a-4097-b750-2733c7fec40f	ORD-0082	AAPL	1	150.82	100	NEW	2026-03-31 14:09:54
12fe52c7-b9d0-4f5d-9bc9-9c47720acf2a	ORD-0060	AAPL	1	150.60	100	NEW	2026-03-31 14:09:53
1ac8509f-ade6-4d07-9627-27148cc419b7	ORD-0012	AAPL	1	150.12	100	NEW	2026-03-31 14:09:53
1ad8ba82-e4a0-41ae-a6b7-763f3334a63c	ORD-0027	AAPL	1	150.27	100	NEW	2026-03-31 14:09:53
1ce9bda6-e403-4d98-83c8-9e1f2afb2b5e	ORD-0094	AAPL	1	150.94	100	NEW	2026-03-31 14:09:54
21483a2b-9034-49ad-ab03-6013a9443518	ORD-0084	AAPL	1	150.84	100	NEW	2026-03-31 14:09:54
2331ecd3-8148-41e0-9c94-8a23fc839b5b	ORD-0090	AAPL	1	150.90	100	NEW	2026-03-31 14:09:54
249ab561-ae3d-4d52-b943-4dfcb8e08246	ORD-0048	AAPL	1	150.48	100	NEW	2026-03-31 14:09:53
296b4986-db61-4335-aa9d-1304be751fb9	ORD-0041	AAPL	1	150.41	100	NEW	2026-03-31 14:09:53
297b0dd5-c7ec-481e-ab4c-6534b957cb9a	ORD-0042	AAPL	1	150.42	100	NEW	2026-03-31 14:09:53

Q.3 Why did we use LinkedBlockingQueue? What would happen if we used a queue with a fixed capacity (e.g., ArrayBlockingQueue) and the database went offline while orders kept coming?

It is unbounded. offer() always returns true and never blocks the FIX engine thread. No matter how slow or unresponsive MySQL gets, the critical path (receive order → send ACK) always runs at memory speed.

What happens with ArrayBlockingQueue when DB goes offline?

ArrayBlockingQueue has a fixed capacity (e.g., 1000 slots). When MySQL goes offline:

- OrderPersister stops draining the queue (every insert fails)
- Orders keep arriving from MiniFIX and filling slots
- Queue hits capacity → offer() returns false
- Orders are silently dropped — client got an ACK but the order was never stored
- When MySQL comes back, those orders are gone forever

You could use put() instead of offer() to block rather than drop — but then the FIX engine thread stalls waiting for queue space, which defeats the entire async design and causes latency to spike.

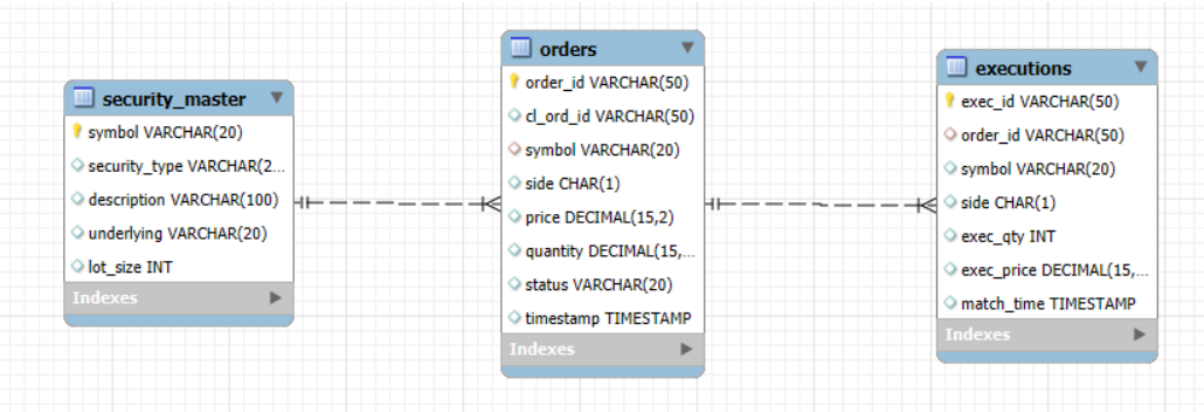
LinkedBlockingQueue trades memory for availability, orders pile up in RAM instead of being dropped, and the FIX engine never stalls. The downside is if the DB is offline long enough, you eventually run out of heap memory.

Ayush Das CML Week 6 Submission

~230958124

~Roll No. 56

Q.1 A simple Entity-Relationship diagram showing how Orders, Executions, and Security Master relate.



Q.2 The Java code snippet showing how you preload the Security Master data into memory at application startup.

```java
public OrderApplication(BlockingQueue<Order> dbQueue) {
    this.dbQueue = dbQueue;
    // Load securities at construction time so they're ready before any order arrives
validSecurities.putAll(DatabaseManager.loadSecurityMaster());
}
```

```java
public static Map<String, Security> loadSecurityMaster() {
    Map<String, Security> map = new HashMap<>();
    String sql = "SELECT symbol, security_type, description, lot_size FROM security_master";
    try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
         Statement stmt  = conn.createStatement();
         ResultSet rs    = stmt.executeQuery(sql)) {

        while (rs.next()) {
            String symbol       = rs.getString(columnLabel: "symbol");
            String securityType = rs.getString(columnLabel: "security_type");
            String description  = rs.getString(columnLabel: "description");
            int    lotSize      = rs.getInt(columnLabel: "lot_size");
            map.put(symbol, new Security(symbol, securityType, description, lotSize));
        }
        System.out.println("[DatabaseManager] Loaded " + map.size() + " securities from security_master.");

    } catch (SQLException e) {
        System.err.println("[DatabaseManager] Failed to load security master: " + e.getMessage());
    }
    return map;
}
```

Q.3 The raw SQL used to create the executions table.

DatabaseManager.java | lab9_runbook.md | FIX.4.4-EXEC_SERVER-MINIFIX_CLIENT

schema.sql

```sql
USE trading_system;

CREATE TABLE IF NOT EXISTS security_master (
    symbol VARCHAR(20) NOT NULL,
    security_type VARCHAR(20) NOT NULL DEFAULT 'CS',
    description VARCHAR(100),
    underlying VARCHAR(20),
    lot_size INT NOT NULL DEFAULT 1,
    CONSTRAINT pk_security PRIMARY KEY (symbol)
);

CREATE TABLE IF NOT EXISTS customer_master (
    customer_code VARCHAR(20) NOT NULL,
    customer_name VARCHAR(100),
    customer_type VARCHAR(20) NOT NULL DEFAULT 'RETAIL',
    credit_limit DECIMAL(20, 2) NOT NULL DEFAULT 0.00,
    CONSTRAINT pk_customer PRIMARY KEY (customer_code)
);

CREATE TABLE IF NOT EXISTS orders (
    order_id VARCHAR(50) NOT NULL,
    cl_ord_id VARCHAR(50) NOT NULL,
    symbol VARCHAR(20) NOT NULL,
    side CHAR(1) NOT NULL,
    price DECIMAL(15, 2) NOT NULL,
    quantity DECIMAL(15, 0) NOT NULL,
    leaves_qty DECIMAL(15, 0) NOT NULL,
    cum_qty DECIMAL(15, 0) NOT NULL DEFAULT 0,
    status VARCHAR(20) NOT NULL DEFAULT 'NEW',
    customer_code VARCHAR(20),
    timestamp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_order PRIMARY KEY (order_id),
    CONSTRAINT fk_ord_sym FOREIGN KEY (symbol) REFERENCES security_master(symbol),
    CONSTRAINT fk_ord_cust FOREIGN KEY (customer_code) REFERENCES customer_master(customer_code)
);

CREATE TABLE IF NOT EXISTS executions (
    exec_id VARCHAR(50) NOT NULL,
    order_id VARCHAR(50) NOT NULL,
    resting_order_id VARCHAR(50) NOT NULL,
    symbol VARCHAR(20) NOT NULL,
    side CHAR(1) NOT NULL,
    exec_qty DECIMAL(15, 0) NOT NULL,
    exec_price DECIMAL(15, 2) NOT NULL,
    match_time TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_execution PRIMARY KEY (exec_id),
    CONSTRAINT fk_exec_order FOREIGN KEY (order_id) REFERENCES orders(order_id),
    CONSTRAINT fk_exec_rest_order FOREIGN KEY (resting_order_id) REFERENCES orders(order_id)
);
```

```sql
CREATE TABLE executions (
exec_id VARCHAR(50) PRIMARY KEY,
order_id VARCHAR(50),
symbol VARCHAR(20),
side CHAR(1),
exec_qty INT,
exec_price DECIMAL(15, 2),
match_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_order FOREIGN KEY (order_id) REFERENCES orders(order_id)
);
```

Ayush Das CML Week 7 Submission

~230958124

~Roll No. 56

Q.1 Submit the matchOrder logic in a clear Java file

Ans: Code submitted as a Java File in the Assignment Github Repo itself

Q.2 Provide a log trace showing the following sequence to prove Price-Time priority:

```
Problems   Output   Debug Console   Terminal   Ports                    powershell

PS C:\Users\ayush\Desktop\CML> javac -cp "lib\*" src\*.java
PS C:\Users\ayush\Desktop\CML> java -cp "src;lib\*" Lab7Test
--- Scenario: Assessment Trace ---
Submitting: Order{id=6cf895c4-3f4a-4e1f-9519-2336c0455eba, clOrdID=O-1, symbol=MSFT, side=2, price=50.00, qty=100}
Submitting: Order{id=ddbf1b96-d723-46b1-9bd7-8dba9f6f7777, clOrdID=O-2, symbol=MSFT, side=2, price=51.00, qty=100}
Submitting: Order{id=cb1fee7f-262d-40c1-9caf-998b820b337b, clOrdID=O-3, symbol=MSFT, side=1, price=52.00, qty=150}
TRADE EXECUTED: MSFT 100 shares @ $50.00
TRADE EXECUTED: MSFT 50 shares @ $51.00
Remaining Bids in book: {}
Remaining Asks in book: {51.0=[Order{id=ddbf1b96-d723-46b1-9bd7-8dba9f6f7777, clOrdID=O-2, symbol=MSFT, side=2, price=5
1.00, qty=50}]}
PS C:\Users\ayush\Desktop\CML>
```

Ayush Das CML Week 8 Submission

~230958124

~Roll No. 56

Q.1 Submit a screenshot of the **MiniFix Blotter** showing an order with OrdStatus=Filled and ExecType=Trade.

```
14.3ms  8=FIX.4.4|9=251|35=8|34=3|49=EXEC_SERVER|52=20260331-16:27:59.352|56=MINIFIX_CLIENT|6=50|11=ORD001|14=100|17=22085151-9562-472c-85c6-11b81dadc2a5|31=50|32=100|37=7be23a09-ceae-4841-b07a-50a
4.0ms   8=FIX.4.4|9=256|35=8|34=4|49=EXEC_SERVER|52=20260331-16:27:59.355|56=MINIFIX_CLIENT|6=50|11=ORD001|14=100|17=22085151-9562-472c-85c6-11b81dadc2a5-REST|31=50|32=100|37=58638cc0-be1f-468f-a1e
```

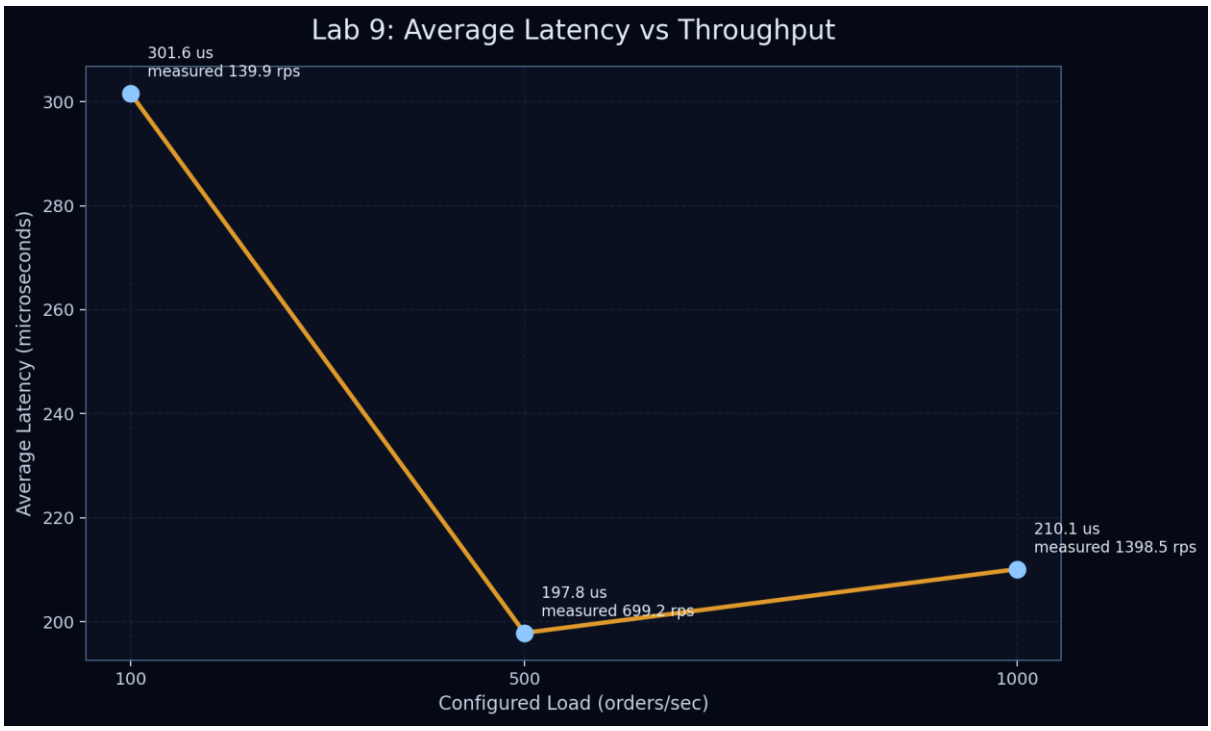
OrderID	AvgPx	ClOrdID	CumQty	ExecID	OrderQty	OrdStatus	Price	Side	Symbol	ExecType	LeavesQty
7be23a09-ceae-4841-b07a-50a94e10a1da	50	ORD001	100	22085151-9562-472c-85c6-11b81dadc...	100	Filled	50	Sell	MSFT	Trade	0
58638cc0-be1f-468f-a1e5-5766a4ca98c9	50	ORD001	100	22085151-9562-472c-85c6-11b81dadc...	100	Filled	50	Buy	MSFT	Trade	0

Ayush Das CML Week 9 Submission

~230958124

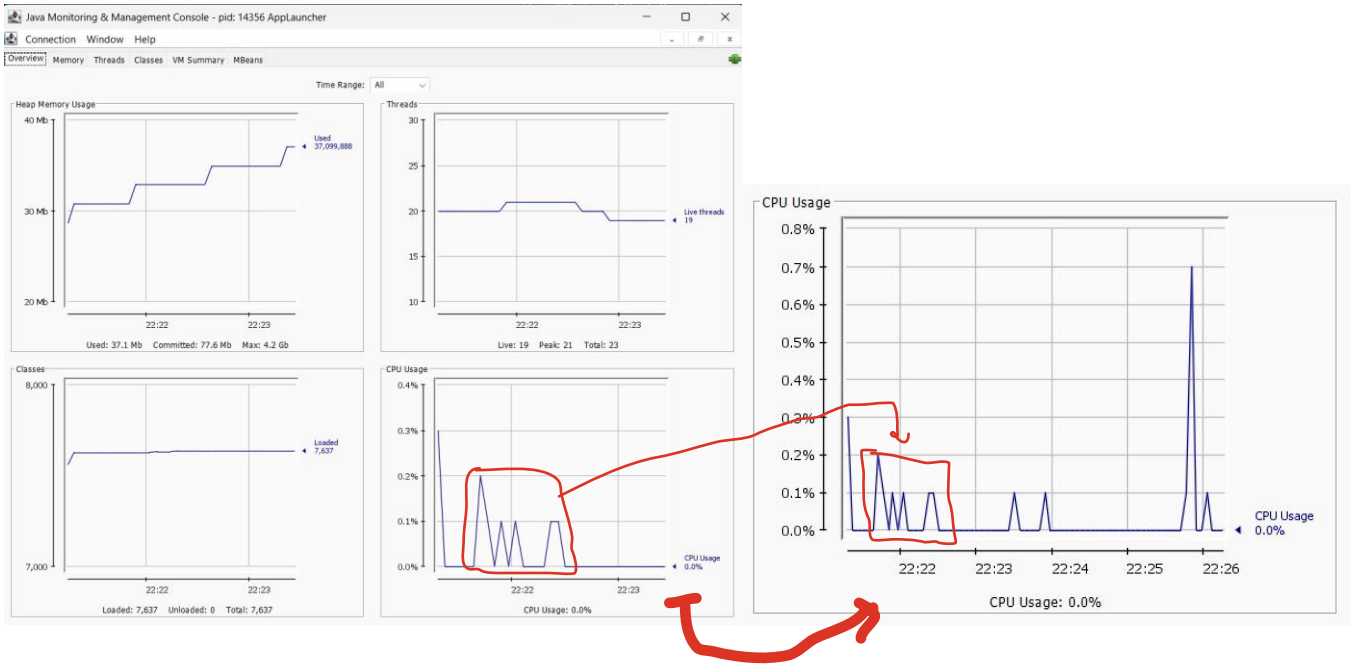
~Roll No. 56

Q.1 A plot (from Excel/Python) of Average Latency vs. Throughput (run tests at 100, 500, and 1000 orders/sec).

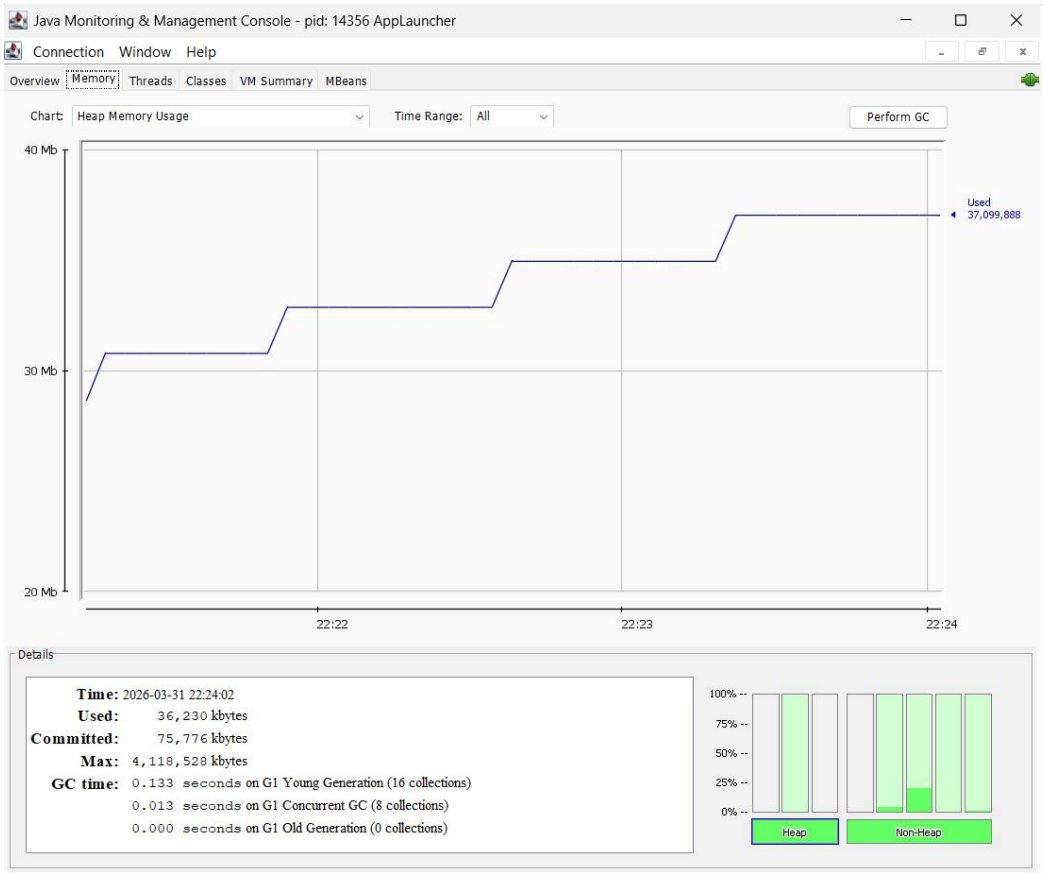


Q.2 Provide a log trace showing the following sequence to prove Price-Time priority:

CPU Usage
Extra Info: Heap Memory, Threads, Classes



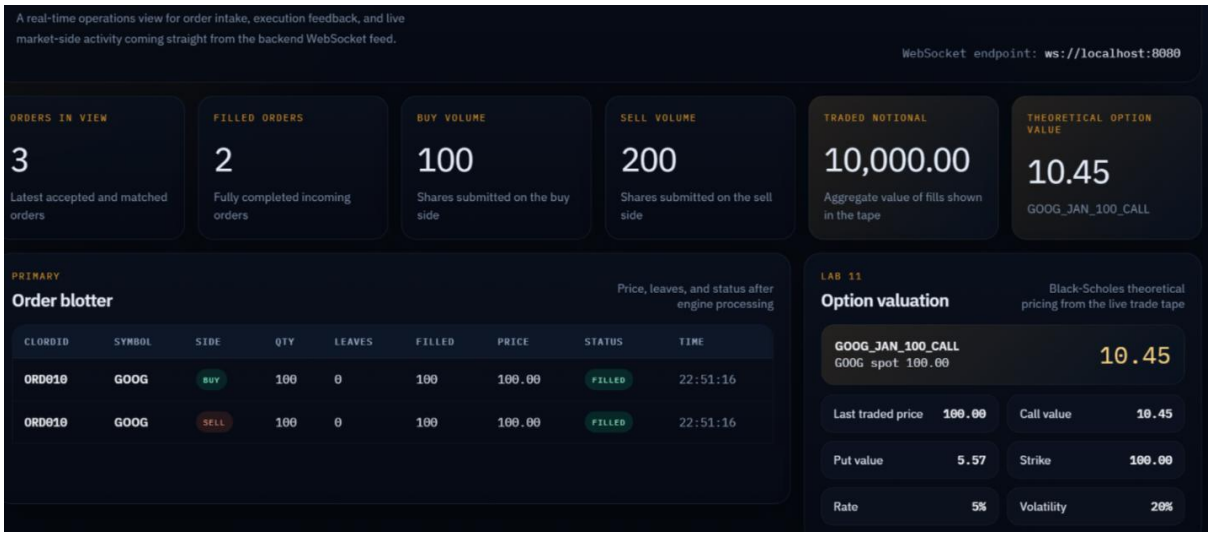
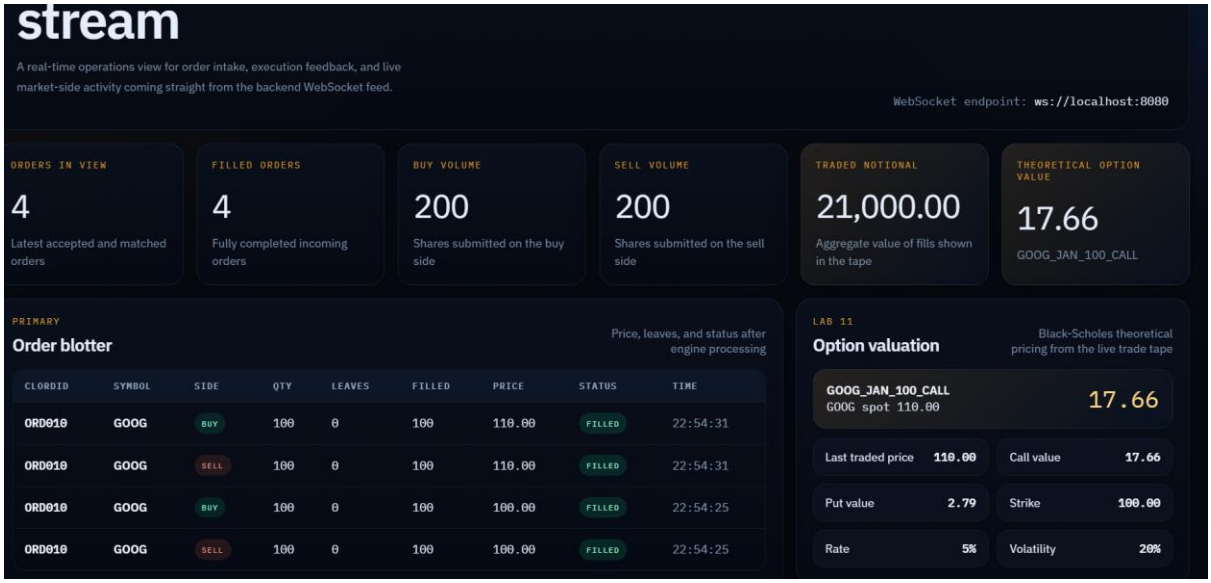
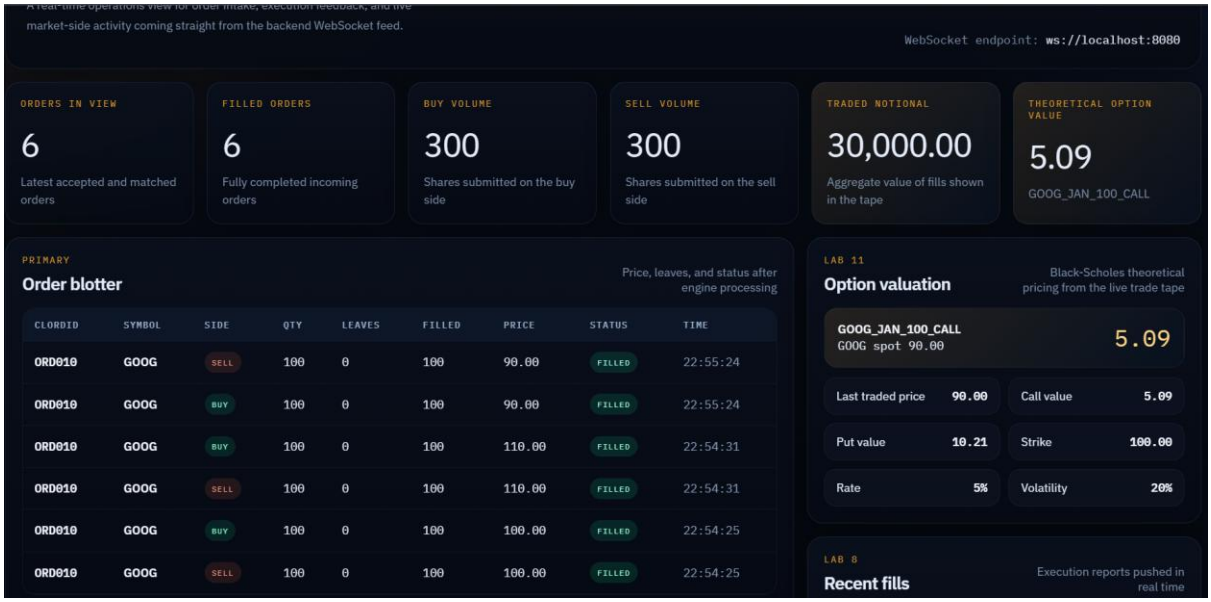
Memory Usage



Q.3 **Conclusion:** What is the bottleneck in your system? (Hint: It is almost certainly the Database or the Console Logging).

The primary bottleneck in the system is the database write path, with console logging as a secondary bottleneck. Even though order persistence is offloaded to a background queue, each order and execution still results in JDBC work and disk I/O, which limits throughput under heavy bursts. When verbose logging is enabled, repeated console printing also adds noticeable overhead and increases latency variance. The matching engine itself is relatively efficient in memory, so the main performance constraints come from persistence and logging rather than the core matching logic.

Ayush Das CML Week 11 Submission

~230958124

~Roll No. 56